
Introduction

This part contains a selection of algorithmic topics that extend and complement earlier material in this book. Some chapters introduce new models of computation such as circuits or parallel computers. Others cover specialized domains such as matrices or number theory. The last two chapters discuss some of the known limitations to the design of efficient algorithms and introduce techniques for coping with those limitations.

Chapter 26 presents an algorithmic model for parallel computing based on task-parallel computing, and more specifically, fork-join parallelism. The chapter introduces the basics of the model, showing how to quantify parallelism in terms of the measures of work and span. It then investigates several interesting fork-join algorithms, including algorithms for matrix multiplication and merge sorting.

An algorithm that receives its input over time, rather than having the entire input available at the start, is called an “online” algorithm. Chapter 27 examines techniques used in online algorithms, starting with the “toy” problem of how long to wait for an elevator before taking the stairs. It then studies the “move-to-front” heuristic for maintaining a linked list and finishes with the online version of the caching problem we saw back in Section 15.4. The analyses of these online algorithms are remarkable in that they prove that these algorithms, which do not know their future inputs, perform within a constant factor of optimal algorithms that know the future inputs.

Chapter 28 studies efficient algorithms for operating on matrices. It presents two general methods—LU decomposition and LUP decomposition—for solving linear equations by Gaussian elimination in $O(n^3)$ time. It also shows that matrix inversion and matrix multiplication can be performed equally fast. The chapter concludes by showing how to compute a least-squares approximate solution when a set of linear equations has no exact solution.

Chapter 29 studies how to model problems as linear programs, where the goal is to maximize or minimize an objective, given limited resources and competing constraints. Linear programming arises in a variety of practical application areas. The chapter also addresses the concept of “duality” which, by establishing that a maximization problem and minimization problem have the same objective value, helps to show that solutions to each are optimal.

Chapter 30 studies operations on polynomials and shows how to use a well-known signal-processing technique—the fast Fourier transform (FFT)—to multiply two degree- n polynomials in $O(n \lg n)$ time. It also derives a parallel circuit to compute the FFT.

Chapter 31 presents number-theoretic algorithms. After reviewing elementary number theory, it presents Euclid’s algorithm for computing greatest common divisors. Next, it studies algorithms for solving modular linear equations and for raising one number to a power modulo another number. Then, it explores an important application of number-theoretic algorithms: the RSA public-key cryptosystem. This cryptosystem can be used not only to encrypt messages so that an adversary cannot read them, but also to provide digital signatures. The chapter finishes with the Miller-Rabin randomized primality test, which enables finding large primes efficiently—an essential requirement for the RSA system.

Chapter 32 studies the problem of finding all occurrences of a given pattern string in a given text string, a problem that arises frequently in text-editing programs. After examining the naive approach, the chapter presents an elegant approach due to Rabin and Karp. Then, after showing an efficient solution based on finite automata, the chapter presents the Knuth-Morris-Pratt algorithm, which modifies the

automaton-based algorithm to save space by cleverly preprocessing the pattern. The chapter finishes by studying suffix arrays, which can not only find a pattern in a text string, but can do quite a bit more, such as finding the longest repeated substring in a text and finding the longest common substring appearing in two texts.

Chapter 33 examines three algorithms within the expansive field of machine learning. Machine-learning algorithms are designed to take in vast amounts of data, devise hypotheses about patterns in the data, and test these hypotheses. The chapter starts with k -means clustering, which groups data elements into k classes based on how similar they are to each other. It then shows how to use the technique of multiplicative weights to make predictions accurately based on a set of “experts” of varying quality. Perhaps surprisingly, even without knowing which experts are reliable and which are not, you can predict almost as accurately as the most reliable expert. The chapter finishes with gradient descent, an optimization technique that finds a local minimum value for a function. Gradient descent has many applications, including finding parameter settings for many machine-learning models.

Chapter 34 concerns NP-complete problems. Many interesting computational problems are NP-complete, but no polynomial-time algorithm is known for solving any of them. This chapter presents techniques for determining when a problem is NP-complete, using them to prove several classic problems NP-complete: determining whether a graph has a hamiltonian cycle (a cycle that includes every vertex), determining whether a boolean formula is satisfiable (whether there exists an assignment of boolean values to its variables that causes the formula to evaluate to TRUE), and determining whether a given set of numbers has a subset that adds up to a given target value. The chapter also proves that the famous traveling-salesperson problem (find a shortest route that starts and ends at the same location and visits each of a set of locations once) is NP-complete.

Chapter 35 shows how to find approximate solutions to NP-complete problems efficiently by using approximation algorithms. For some NP-complete problems, approximate solutions that are near optimal are quite easy to produce, but for others even the best approximation algorithms known work progressively more poorly as the problem size

increases. Then, there are some problems for which investing increasing amounts of computation time yields increasingly better approximate solutions. This chapter illustrates these possibilities with the vertex-cover problem (unweighted and weighted versions), an optimization version of 3-CNF satisfiability, the traveling-salesperson problem, the set-covering problem, and the subset-sum problem.